

What is debugging?

Debugging is an essential part of software development that helps developers identify and fix errors. This ensures that the application runs smoothly without causing errors. **NodeJS** is the JavaScript runtime environment that provides various debugging tools for troubleshooting the application.

What is Node Debugging?

Debugging is the process of finding and fixing errors (bugs) in your code. It's an important part of software development, as almost all code has some bugs at some point.

- They help in identifying runtime errors.
- Improves code efficiency by detecting memory leaks.
- Enables developers to monitor application execution flow.
- Ensures error-free and reliable applications.

methods and tools for debugging:-

Console Logging:

Using **console.log()** to output variable values and track code execution flow is a basic but useful debugging technique.

```
const user = { name: "Alice", age: 25 };  
console.log("User data:", user);
```

- **Pros:**

Quick and easy to use

Works in all environments

- **Cons:**

Can clutter code

Not efficient for complex debugging

Core Node.js Debugger

The built-in Node.js Debugger (node inspect) allows you to step through your code execution, set breakpoints, inspect variables, and debug effectively without third-party tools.

1. Starting the Debugger

```
node --inspect app.js
```

Common Debugger Commands

Command	Description
c	Continue execution until the next breakpoint
n	Move to the next line (step over)
s	Step into the function call
o	Step out of the current function
watch('variable')	Watch a variable's value
repl	Open an interactive REPL for live evaluation
bt	Print the current stack trace
list(n)	Show n lines of code around the current execution point
pause	Pause execution at the current point

```
Example code
function add(a, b) {
  return a + b;
}

function multiply(a, b) {
  return a * b;
}

function main() {
  let x = 5;
  let y = 10;
  let sum = add(x, y);
  let product = multiply(x, y);

  console.log("Sum:", sum);
  console.log("Product:", product);
}

main()
```

Running Debugger Step-by-Step

1. Start Debugging

```
node inspect app.js
```

2. Set a Breakpoint Before Running Code

```
debug> sb(9)
```

This sets a breakpoint at line **9**, where `let sum = add(x, y);` executes.

3. Continue Execution Until Breakpoint

```
debug> c
```

The program runs until it reaches line **9**.

4. Step into the `add()` Function

```
debug> s
```

Now you're inside the `add()` function. You can inspect variable values.

5. Check Variable Values

```
debug> repl
```

Inside REPL mode, type:

```
x  
y
```

This will return 5 and 10.

6. Continue Execution

```
debug> c
```

The program resumes execution.

5. Debugging with Breakpoints and Watchers

You can also use **watch expressions** to monitor variables:

```
debug> watch('sum')  
debug> watch('product')
```

This will show the values of `sum` and `product` whenever they change.

6. Exiting the Debugger

To exit the debugger, use:

```
debug> .exit
```

